

Q1. Write Java code to connect to MySQL using JDBC.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class MySQLConnectionDemo {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/college_db";
        String user = "root";
        String password = "password";

        try (Connection conn = DriverManager.getConnection(url, user, password)) {
            System.out.println("Connected to database successfully.");
        } catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }
    }
}
```

This program uses `DriverManager.getConnection` with the database URL, username, and password to establish a JDBC connection to a MySQL database named `college_db`. The try-with-resources syntax ensures the connection is automatically closed once the block finishes, even if an error occurs.

Q2. Execute an INSERT and SELECT statement using JDBC.

```
import java.sql.*;

public class JdbcInsertSelectDemo {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/college_db";
        String user = "root";
        String password = "password";

        try (Connection conn = DriverManager.getConnection(url, user, password);
            Statement stmt = conn.createStatement()) {

            // INSERT statement
            String insertSql =
```

```

        "INSERT INTO students (name, marks) VALUES ('Ravi', 85)";
stmt.executeUpdate(insertSql);
System.out.println("Record inserted.");

// SELECT statement
String selectSql = "SELECT name, marks FROM students";
ResultSet rs = stmt.executeQuery(selectSql);

while (rs.next()) {
    System.out.println(rs.getString("name") + " - " + rs.getInt("marks"));
}

} catch (SQLException e) {
    System.out.println("Database error: " + e.getMessage());
}
}
}

```

executeUpdate is used for the INSERT statement, since it modifies the database and returns the number of affected rows. executeQuery is used for the SELECT statement, since it retrieves data and returns a ResultSet, which is then iterated row by row using rs.next() to read and print each student's name and marks.

Q3. Explain the purpose of prepared statements in Java.

A PreparedStatement is a precompiled SQL statement that uses placeholders (question marks) for values that are supplied separately, rather than being concatenated directly into the SQL string.

```

import java.sql.*;

public class PreparedStatementDemo {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/college_db";
        String user = "root";
        String password = "password";

        String sql = "INSERT INTO students (name, marks) VALUES (?, ?)";

        try (Connection conn = DriverManager.getConnection(url, user, password);
            PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setString(1, "Anita");
            ps.setInt(2, 92);
        }
    }
}

```

```
        ps.executeUpdate();

        System.out.println("Record inserted using PreparedStatement.");

    } catch (SQLException e) {
        System.out.println("Database error: " + e.getMessage());
    }
}
}
```

PreparedStatement serves several important purposes:

- **Security:** it protects against SQL injection attacks, since user-supplied values are safely bound to placeholders rather than being directly inserted into the SQL text.
- **Performance:** the SQL statement can be precompiled once by the database and reused efficiently with different parameter values across multiple executions.
- **Convenience and correctness:** type-safe setter methods, such as `setString` and `setInt`, handle proper formatting and escaping of values automatically, reducing the chance of syntax errors.

For these reasons, PreparedStatement is generally preferred over a plain Statement whenever a SQL query includes values supplied by user input or external data.